

Plan de Mejora Continua y Verificación de Software — Proyecto Lubix

Jose Armenta Farias

Johan Nicolás Acuña Duarte

Juan David Molina Sánchez

Daniel Matias Monroy Huertas

Centro de Electricidad, Electrónica y Telecomunicaciones (CEET)

Servicio Nacional de Aprendizaje (SENA)

Análisis y Desarrollo de Software (ADSO)

31 de agosto de 2026

PLAN DE MEJORA CONTINUA Y VERIFICACIÓN DE SOFTWARE

LUBIX — Plataforma de Gestión Empresarial

1. Introducción

La Verificación de Software es la fase del desarrollo donde se ejecutan pruebas sobre los componentes del sistema para comprobar que funcionan correctamente y cumplen los requisitos. En este proyecto se verificaron los endpoints del backend, la lógica de negocio (autenticación, carrito, pedidos, productos) y la integración con la base de datos.

A partir de los resultados de estas verificaciones, se estructuró un Plan de Mejora Continua que organiza los hallazgos en tres tipos de acciones según la metodología de calidad de software:

- **Acciones Correctivas:** Corregir errores ya detectados durante la fase de evaluación.
- **Acciones Preventivas:** Evitar que los fallos vuelvan a ocurrir o se produzcan nuevos riesgos en producción.
- **Acciones de Ajuste:** Optimizar el rendimiento general, la mantenibilidad y la arquitectura del sistema.

2. Verificación de Software en Lubix

Durante la verificación se aplicaron pruebas exhaustivas sobre los siguientes módulos operativos:

- **Autenticación:** Registro de usuario y empresa, verificación por correo con código de 6 dígitos, inicio de sesión con JWT, recuperación y restablecimiento de contraseña, tokens de refresco y cierre de sesión.

- **Carrito de compras:** Agregar, modificar, eliminar y vaciar productos; cálculo de subtotales; validación de stock; fusión de carrito local con el del servidor.
- **Catálogo y productos:** Búsqueda con filtros, ordenamiento, detalle de producto, productos relacionados, reseñas y calificación promedio.
- **Pedidos:** Creación de pedido, validación de stock, historial, detalle, cancelación y descarga de factura.
- **Módulo administrativo:** Panel con KPIs, gestión de usuarios, validación de empresas y resolución de PQRS.
- **Base de datos:** Integridad de las relaciones entre las tablas mediante ORM (SQLAlchemy) y migraciones con Alembic.

Para llevar a cabo estas evaluaciones de manera rigurosa, se emplearon las siguientes herramientas de verificación:

- **Documentación interactiva de la API en Swagger (/docs):** Para probar y validar el comportamiento de cada endpoint en tiempo real.
- **Pruebas manuales de integración:** Verificación entre el backend en FastAPI, el frontend web en React y la aplicación móvil en React Native/Expo.
- **pip-audit:** Para la detección automatizada de vulnerabilidades conocidas (CVE) en las dependencias de Python.
- **Validaciones de esquemas con Pydantic:** Para garantizar la consistencia y el tipado correcto de las entradas y salidas de la API REST.

3. Plan de Mejora Continua

3.1. Acciones Correctivas — Errores Solucionados

Las acciones correctivas atienden fallos que se presentaron y fueron corregidos durante o después de la verificación técnica.

Tabla 1

Matriz de Acciones Correctivas Aplicadas en Lubix

#	Error Detectado	Corrección Aplicada
C1	Lentitud en la carga del carrito y del detalle de producto debido a múltiples consultas repetidas a la base de datos (problema N+1).	Se optimizaron las consultas con SQLAlchemy (uso de joinedload y consultas con join), reduciendo las llamadas a la base de datos y mejorando el tiempo de respuesta.
C2	Stock insuficiente permitido al agregar o modificar cantidades en el carrito.	Se implementó la validación de stock en los endpoints del carrito, devolviendo un error 400 con el mensaje "Stock insuficiente" cuando la cantidad excede lo disponible.
C3	Un usuario podía reseñar múltiples veces un mismo producto.	Se agregó una restricción única (UniqueConstraint) sobre la combinación user_id y product_id en la tabla reviews, y se validó en el backend que el usuario solo pueda dejar una reseña por producto.

C4	Duplicado de códigos o productos en el carrito.	Se aplicaron restricciones únicas en <code>cart_items</code> (carrito + producto) y se implementó la lógica de incrementar la cantidad en lugar de crear líneas duplicadas.
C5	Debilidad en el borrado de datos relacionados (carrito, pedidos, reseñas, favoritos).	Se configuraron relaciones con <code>cascade="all, delete-orphan"</code> para garantizar un borrado coherente y evitar datos huérfanos en la base de datos.

Nota. Elaboración propia basada en las pruebas de integración del backend de Lubix.

3.2. Acciones Preventivas — Evitar Fallos Futuros

Las acciones preventivas buscan anticiparse a los problemas y proteger la integridad, la seguridad y la disponibilidad del sistema.

Tabla 2

Matriz de Acciones Preventivas Implementadas en Lubix

#	Riesgo o Fallo Potencial	Medida Preventiva
P1	Colapso del almacenamiento MinIO al subir archivos pesados (logo, banner o certificado de empresa).	Se validó el tipo de archivo y se estableció un límite de tamaño dentro del servicio de almacenamiento (NasService) para rechazar archivos no permitidos.
P2	Ataques de fuerza bruta en el inicio de sesión.	Uso de contraseñas cifradas con <code>bcrypt/argon2</code> y tokens JWT con

		expiración, junto con tokens de refresco revocables en la tabla refreshToken.
P3	Acceso no autorizado a recursos del usuario.	Middleware de autenticación que valida el JWT en cada petición protegida y CORS configurado para controlar los orígenes permitidos.
P4	Códigos de verificación reutilizables o vencidos.	Los códigos de verificación y de restablecimiento de contraseña tienen una expiración de 15 minutos y un tipo definido (verifyEmail / resetPassword), evitando su reutilización fuera de tiempo.
P5	Vulnerabilidades en dependencias.	Se integró pip-audit en el flujo de desarrollo para auditar los paquetes e identificar vulnerabilidades CVE, actualizando las librerías afectadas antes de su despliegue.
P6	Productos inactivos o sin stock que se exhiben a los clientes.	En las consultas públicas del catálogo se filtra por status = "active" y se valida el stock en tiempo real al agregar productos al carrito o crear el pedido.

Nota. Elaboración propia basada en las políticas de seguridad y arquitectura de Lubix.

3.3. Acciones de Ajuste — Mejoras Operativas y de Arquitectura

Las acciones de ajuste optimizan el rendimiento, la mantenibilidad y la experiencia del usuario sin necesariamente corregir un error explícito.

Tabla 3

Matriz de Acciones de Ajuste y Mantenibilidad en Lubix

#	Área	Ajuste Aplicado
A1	Estandarización de respuestas	Se organizaron los routers del backend (/auth, /user, /cart, /company, /admin, /products, /pqrs) definiendo esquemas de entrada con Pydantic, garantizando una estructura de respuesta uniforme que consume el frontend web y la app móvil.
A2	Arquitectura modular	Separación de responsabilidades en el backend: routers (rutas), services (lógica de negocio), schemas (validación) y models (ORM). Esto facilita mantener y ampliar cada módulo de forma independiente.
A3	Rendimiento del panel de administración	El dashboard de administración agrupa los indicadores clave (total de usuarios, empresas, empresas pendientes, usuarios activos/inactivos) en una sola consulta agregada, reduciendo la carga sobre la base de datos.
A4	Migraciones controladas	Se centralizó el control del esquema de base de datos con Alembic, permitiendo versionar los cambios y

		aplicar migraciones de forma ordenada en cualquier entorno.
A5	Sincronización de carrito	Se implementó el endpoint de fusión de carrito (/merge) que combina el carrito local del visitante con el carrito del servidor al iniciar sesión, mejorando la experiencia del usuario sin perder productos seleccionados.
A6	Configuración por variables de entorno	Centralización de la configuración en el archivo .env (base de datos, JWT, SMTP, MinIO) para facilitar el despliegue en distintos entornos (desarrollo, test, producción).

Nota. Elaboración propia basada en la estructura de código del repositorio Lubix.

4. Explicación Verbal (Guion para la Sustentación)

A continuación, se presenta el guion estructurado para la sustentación oral ante el jurado evaluador del SENA:

«Durante la fase de verificación de software realizamos pruebas a los endpoints, a la base de datos y a la integración del frontend web y la app móvil de Lubix. A partir de los resultados, estructuramos un Plan de Mejora Continua organizado en tres ejes:

Primero, las acciones correctivas, con las que solucionamos errores detectados, como la lentitud en la carga del carrito que optimizamos mejorando las consultas a la base de datos, o la validación del stock para evitar que se agreguen

cantidades superiores a las disponibles.

Segundo, las acciones preventivas, orientadas a evitar fallos futuros, como la validación del tamaño y formato de los archivos para proteger el almacenamiento en MinIO, la expiración de los códigos de verificación, y el uso de contraseñas cifradas y tokens JWT para proteger la autenticación.

Y tercero, las acciones de ajuste, con las que mejoramos la arquitectura y el rendimiento, como la estandarización de las respuestas JSON mediante esquemas de Pydantic, la organización modular del backend y la optimización de las consultas del panel de administración.»

5. Conclusión

La verificación de software permitió detectar y corregir fallos reales en el funcionamiento de Lubix. El Plan de Mejora Continua garantiza que el proyecto no solo funcione en el presente, sino que sea mantenible, seguro y escalable a futuro. Al clasificar cada acción en correctivas, preventivas y de ajuste, se demuestra un criterio claro y profesional en la gestión de calidad de software exigida por los estándares de la industria y la formación ADSO.